SonarQube Analysis Report

LabReserveQA
Generated on October 30, 2026 at 01:31

Issue Type	Count
Critical Issues	0
Major Issues	6
Minor Issues	1
Info Issues	0
Code Smells	2
Bugs	3
Vulnerabilities	2

Detailed Issue Analysis

Issue Details

ISSUE #1

● *MAJOR* | *CODE_SMELL*

Simplify this regular expression to reduce its runtime, as it has super-linear performance due to backtracking.

■ *Where is this issue?*

File: server/config/db.js
Line: 6

■ *Why is this an issue?*

Regular expression engines use backtracking to try all possible execution paths when evaluating a pattern against an input. In some cases, this leads to non-linear backtracking where the worst-case evaluation time grows polynomially (e.g., $O(n^2)$ or $O(n^3)$) with the input size. While not as severe as catastrophic backtracking, such patterns can significantly degrade application performance when processing large or untrusted inputs.

This rule reports regular expressions that exhibit non-linear backtracking behavior.

■ *How can I fix it?*

To fix a regular expression with non-linear backtracking, consider the following strategies:

• Replace `.` with negated character classes to exclude separators where applicable (e.g., `[^,]*` instead of `.*` before `,`).

• Use bounded quantifiers such as `{1,5}` to limit repetitions.

- Restructure alternations and quantifiers to eliminate ambiguity — avoid patterns where multiple alternatives can match the same character.
- JavaScript does not support possessive quantifiers, but atomic grouping can be emulated using lookahead assertions and backreferences.
- Use a non-backtracking implementation such as node-re2.

The following examples illustrate two distinct causes of non-linear backtracking. The first shows a pattern that is polynomial when unanchored: without a start anchor, the engine retries the pattern at every position, leading to quadratic evaluation time when there is no match. The second shows ambiguous alternation, which causes quadratic backtracking regardless of anchoring because multiple alternatives can match the same input in different ways.

Noncompliant code example

■ *Noncompliant Code Example:*

```
1 ■ /a+b/.test(input); // Noncompliant - polynomial backtracking when the pattern does not match <<<
```

■ *Noncompliant Code Example:*

```
1 ■ /(a|aa)+/.test(input); // Noncompliant - ambiguous alternation causes quadratic backtracking <<<
```

✓ *Compliant Solution:*

```
1 ■ /^a+b/.test(input); // Compliant - anchor eliminates redundant backtracking positions <<<
```

✓ *Compliant Solution:*

```
1 ■ /a+/.test(input); // Compliant - unambiguous <<<
```

ISSUE #2

● *MAJOR* | *CODE_SMELL*

Simplify this regular expression to reduce its runtime, as it has super-linear performance due to backtracking.

■ *Where is this issue?*

File: server/models/User.js
Line: 17

■ *Why is this an issue?*

Regular expression engines use backtracking to try all possible execution paths when evaluating a pattern against an input. In some cases, this leads to non-linear backtracking where the worst-case evaluation time grows polynomially (e.g., $O(n^2)$ or $O(n^3)$) with the input size. While not as severe as catastrophic backtracking, such patterns can significantly degrade application performance when processing large or untrusted inputs.

This rule reports regular expressions that exhibit non-linear backtracking behavior.

■ *How can I fix it?*

To fix a regular expression with non-linear backtracking, consider the following strategies:

• Replace `.` with negated character classes to exclude separators where applicable (e.g., `[^,]*` instead of `.*` before `,`).

• Use bounded quantifiers such as `{1,5}` to limit repetitions.

- Restructure alternations and quantifiers to eliminate ambiguity — avoid patterns where multiple alternatives can match the same character.
- JavaScript does not support possessive quantifiers, but atomic grouping can be emulated using lookahead assertions and backreferences.
- Use a non-backtracking implementation such as node-re2.

The following examples illustrate two distinct causes of non-linear backtracking. The first shows a pattern that is polynomial when unanchored: without a start anchor, the engine retries the pattern at every position, leading to quadratic evaluation time when there is no match. The second shows ambiguous alternation, which causes quadratic backtracking regardless of anchoring because multiple alternatives can match the same input in different ways.

Noncompliant code example

■ *Noncompliant Code Example:*

```
1 ■ /a+b/.test(input); // Noncompliant - polynomial backtracking when the pattern does not match <<<
```

■ *Noncompliant Code Example:*

```
1 ■ /(a|aa)+/.test(input); // Noncompliant - ambiguous alternation causes quadratic backtracking <<<
```

✓ *Compliant Solution:*

```
1 ■ /^a+b/.test(input); // Compliant - anchor eliminates redundant backtracking positions <<<
```

✓ *Compliant Solution:*

```
1 ■ /a+/.test(input); // Compliant - unambiguous <<<
```

ISSUE #3

● ***MAJOR*** | ***BUG***

Promises must be awaited, end with a call to .catch, end with a call to .then with a rejection handler or be explicitly marked as ignored with the `void` operator.

■ *Where is this issue?*

File: server/scripts/seedAdmin.js
Line: 48

■ *Why is this an issue?*

A Promise represents the eventual result of an asynchronous operation. When a Promise is created but neither awaited, returned, nor given a rejection handler, it is said to be "floating": nothing in the code observes whether it eventually resolves or rejects.

This is a frequent source of bugs:

- If the Promise rejects, nothing in the code reacts to the failure. The rejection escapes to the runtime's global handler instead of the error handling around the call site, so recovery, retries, and user-facing messages never run.
- Operations that were meant to happen in a specific order may instead run concurrently or out of order, because the code moved on before the asynchronous operation completed. Code that depends on the result may then execute too early, using stale or default data.
- An array of Promises, such as the result of a `map()` call, is just as easy to leave unhandled as a single Promise, and is a common way to accidentally launch several operations without ever finding out if any of them failed.

This rule applies to Promise-valued statements. Promises used where another type is expected, such as in a conditional or a spread, are covered by S6544.

What is the potential impact?

An unhandled rejection is not contained to the operation that failed. Since Node.js 15, the default `--unhandled-rejections` mode is `throw`, which raises the rejection as an uncaught exception and terminates the process unless an `unhandledRejection` hook is registered. A single unobserved background task can therefore take down a server that was otherwise healthy.

■ *How can I fix it?*

Make sure every Promise-returning statement is handled in one of the following ways:

- `await` it.
- `return` it, so that the caller becomes responsible for handling it.
- Attach a rejection handler with `.catch()`, or pass a rejection handler as the second argument to `.then()`.
- If you are dealing with several Promises at once, combine them with `Promise.all()`, `Promise.allSettled()`, `Promise.race()`, or `Promise.any()` before handling the result, instead of leaving an array of Promises unhandled.

If a Promise's outcome genuinely does not matter and you want to make that explicit, use the `void` operator on the expression. This documents that the Promise is intentionally left unobserved. Note that this does not make rejections disappear: it only silences this rule, so it should be reserved for Promises that you know cannot practically reject, or that are otherwise covered by a global unhandled-rejection handler.

Noncompliant code example

■ Noncompliant Code Example:

```
1 ■ function sendMetrics(payload) { <<<
2 ■   fetch('/metrics', { method: 'POST', body: payload }); // Noncompliant: nothing handles a failed request
3 ■ }
```

✓ Compliant Solution:

```
1 ■ async function sendMetrics(payload) { <<<
2 ■   await fetch('/metrics', { method: 'POST', body: payload });
3 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs().then(logs =&gt; storeLogs(logs)); // Noncompliant: no rejection handler
3 ■ }
```

✓ Compliant Solution:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs()
3 ■     .then(logs =&gt; storeLogs(logs))
4 ■     .catch(error =&gt; reportError(error));
5 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function notifyAll(subscribers) { <<<
```

```
  subscribers.map(subscriber =&gt; subscriber.notify()); // Noncompliant: an array of unhandled Promises
}
```

✓ Compliant Solution:

```
async function notifyAll(subscribers) { <<<
  await Promise.all(subscribers.map(subscriber =&gt; subscriber.notify()));
}
```

■ Noncompliant Code Example:

```
function onClick() { <<<
  trackEvent('button_clicked'); // Noncompliant: the Promise is left unhandled and is not marked as intentionally ignore ■ d
}
```

✓ Compliant Solution:

```
function onClick() { <<<
  // The analytics call is not expected to fail in a way that matters here.
  void trackEvent('button_clicked');
}
```

ISSUE #4

● ***MAJOR*** | ***BUG***

Promises must be awaited, end with a call to .catch, end with a call to .then with a rejection handler or be explicitly marked as ignored with the `void` operator.

■ *Where is this issue?*

File: server/scripts/seedEquipment.js
Line: 125

■ *Why is this an issue?*

A Promise represents the eventual result of an asynchronous operation. When a Promise is created but neither awaited, returned, nor given a rejection handler, it is said to be "floating": nothing in the code observes whether it eventually resolves or rejects.

This is a frequent source of bugs:

- If the Promise rejects, nothing in the code reacts to the failure. The rejection escapes to the runtime's global handler instead of the error handling around the call site, so recovery, retries, and user-facing messages never run.
- Operations that were meant to happen in a specific order may instead run concurrently or out of order, because the code moved on before the asynchronous operation completed. Code that depends on the result may then execute too early, using stale or default data.
- An array of Promises, such as the result of a `map()` call, is just as easy to leave unhandled as a single Promise, and is a common way to accidentally launch several operations without ever finding out if any of them failed.

This rule applies to Promise-valued statements. Promises used where another type is expected, such as in a conditional or a spread, are covered by S6544.

What is the potential impact?

An unhandled rejection is not contained to the operation that failed. Since Node.js 15, the default `--unhandled-rejections` mode is `throw`, which raises the rejection as an uncaught exception and terminates the process unless an `unhandledRejection` hook is registered. A single unobserved background task can therefore take down a server that was otherwise healthy.

■ *How can I fix it?*

Make sure every Promise-returning statement is handled in one of the following ways:

- `await` it.
- `return` it, so that the caller becomes responsible for handling it.
- Attach a rejection handler with `.catch()`, or pass a rejection handler as the second argument to `.then()`.
- If you are dealing with several Promises at once, combine them with `Promise.all()`, `Promise.allSettled()`, `Promise.race()`, or `Promise.any()` before handling the result, instead of leaving an array of Promises unhandled.

If a Promise's outcome genuinely does not matter and you want to make that explicit, use the `void` operator on the expression. This documents that the Promise is intentionally left unobserved. Note that this does not make rejections disappear: it only silences this rule, so it should be reserved for Promises that you know cannot practically reject, or that are otherwise covered by a global unhandled-rejection handler.

Noncompliant code example

■ Noncompliant Code Example:

```
1 ■ function sendMetrics(payload) { <<<
2 ■   fetch('/metrics', { method: 'POST', body: payload }); // Noncompliant: nothing handles a failed request
3 ■ }
```

✓ Compliant Solution:

```
1 ■ async function sendMetrics(payload) { <<<
2 ■   await fetch('/metrics', { method: 'POST', body: payload });
3 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs().then(logs =&gt; storeLogs(logs)); // Noncompliant: no rejection handler
3 ■ }
```

✓ Compliant Solution:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs()
3 ■     .then(logs =&gt; storeLogs(logs))
4 ■     .catch(error =&gt; reportError(error));
5 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function notifyAll(subscribers) { <<<
```

```
  subscribers.map(subscriber =&gt; subscriber.notify()); // Noncompliant: an array of unhandled Promises
}
```

✓ Compliant Solution:

```
async function notifyAll(subscribers) { <<<
  await Promise.all(subscribers.map(subscriber =&gt; subscriber.notify()));
}
```

■ Noncompliant Code Example:

```
function onClick() { <<<
  trackEvent('button_clicked'); // Noncompliant: the Promise is left unhandled and is not marked as intentionally ignore ■ d
}
```

✓ Compliant Solution:

```
function onClick() { <<<
  // The analytics call is not expected to fail in a way that matters here.
  void trackEvent('button_clicked');
}
```

ISSUE #5

● *MINOR* | *VULNERABILITY*

This framework implicitly discloses version information by default. Make sure it is safe here.

■ *Where is this issue?*

File: server/server.js
Line: 10

■ *Why is this an issue?*

Version information disclosed by default through HTTP headers is often overlooked by developers, yet it can pose a security risk. Once this information is public, attackers can use it to identify potential vulnerabilities specific to that version. This rule raises an issue when version information is disclosed through HTTP headers such as `x-powered-by` or `Server`.

What is the potential impact?

If the disclosed version information indicates the use of outdated or unsupported software, it becomes easier for attackers to exploit known vulnerabilities. They can search for published vulnerabilities related to that version and launch targeted attacks.

■ *How can I fix it?*

In Express.js, version information is disclosed by default in the `x-powered-by` HTTP header. It should be disabled with `app.disable` or with helmet's hidePoweredBy middleware.

Noncompliant code example

■ *Noncompliant Code Example:*

```
1 ■ let express = require('express'); <<<
2 ■
3 ■ let example = express(); // Noncompliant
4 ■
5 ■ example.get('/', function (req, res) {
6 ■   res.send('example')
7 ■ });
```

✓ *Compliant Solution:*

```
1 ■ let express = require('express'); <<<
2 ■
3 ■ let example = express();
4 ■ example.disable("x-powered-by");
```

ISSUE #6

● *MAJOR* | *VULNERABILITY*

Make sure that enabling CORS is safe here.

■ *Where is this issue?*

File: server/server.js
Line: 14

■ *Why is this an issue?*

Same-origin policy in browsers prevents JavaScript from making cross-origin HTTP requests to resources with a different origin (domain, protocol, or port). The Cross-Origin Resource Sharing (CORS) mechanism allows servers to relax this restriction by including `Access-Control-Allow-Origin` response headers that tell browsers which origins are permitted.

Setting the `Access-Control-Allow-Origin` header to a wildcard (`*`) or dynamically reflecting a user-supplied `Origin` header without validation completely disables same-origin protection for the affected resource.

What is the potential impact?

Sensitive data exposure

When CORS restrictions are disabled, a malicious website visited by an authenticated user can issue cross-origin requests to the vulnerable application and read the responses. This allows attackers to steal sensitive data accessible to the victim, such as account details, API keys, or private application data.

Account takeover

If the application is also configured with `Access-Control-Allow-Credentials: true`, the browser will include cookies and HTTP authentication headers in cross-origin requests. Attackers can then perform authenticated operations on behalf of the victim, potentially leading to full account takeover or unauthorized data modification.

■ *How can I fix it?*

Configure the `cors` middleware with specific allowed origins rather than using the default wildcard.

Noncompliant code example

■ *Noncompliant Code Example:*

```
1 ■ const cors = require('cors'); <<<
2 ■
3 ■ let app1 = express();
4 ■ app1.use(cors()); // Noncompliant: by default origin is set to *
5 ■
6 ■ let corsOptions = {
7 ■   origin: '*' // Noncompliant
8 ■ };
9 ■
10 ■ let app2 = express();
11 ■ app2.use(cors(corsOptions));
```

✓ *Compliant Solution:*

```
1 ■ const cors = require('cors'); <<<
2 ■
3 ■ let corsOptions = {
```

```
4 ■   origin: 'trustedwebsite.com'
5 ■ };
6 ■
7 ■ let app = express();
8 ■ app.use(cors(corsOptions));
```

Set the `Access-Control-Allow-Origin` header to a specific trusted origin, and validate user-supplied origins against an allow-list.

Noncompliant code example

■ *Noncompliant Code Example:*

```
1 ■ const http = require('http'); <<<
2 ■ const srv = http.createServer((req, res) =&gt; {
3 ■   res.writeHead(200, { 'Access-Control-Allow-Origin': '*' }); // Noncompliant
4 ■   res.end('ok');
5 ■ });
6 ■ srv.listen(3000);
```

■ *Noncompliant Code Example:*

```
1 ■ function (req, res) { <<<
2 ■   const origin = req.headers.origin;
3 ■   res.setHeader('Access-Control-Allow-Origin', origin); // Noncompliant
4 ■ };
```

✓ *Compliant Solution:*

```
1 ■ const http = require('http'); <<<
2 ■ const srv = http.createServer((req, res) =&gt; {
```

```
  res.writeHead(200, { 'Access-Control-Allow-Origin': 'trustedwebsite.com' });
  res.end('ok');
});
srv.listen(3000);
```

✓ Compliant Solution:

```
function (req, res) { <<<
  const origin = req.headers.origin;

  if (origin === 'https://trustedwebsite.com') {
    res.setHeader('Access-Control-Allow-Origin', origin);
  }
};
```

ISSUE #7

● ***MAJOR*** | ***BUG***

Promises must be awaited, end with a call to .catch, end with a call to .then with a rejection handler or be explicitly marked as ignored with the `void` operator.

■ *Where is this issue?*

File: server/server.js
Line: 50

■ *Why is this an issue?*

A Promise represents the eventual result of an asynchronous operation. When a Promise is created but neither awaited, returned, nor given a rejection handler, it is said to be "floating": nothing in the code observes whether it eventually resolves or rejects.

This is a frequent source of bugs:

- If the Promise rejects, nothing in the code reacts to the failure. The rejection escapes to the runtime’s global handler instead of the error handling around the call site, so recovery, retries, and user-facing messages never run.
- Operations that were meant to happen in a specific order may instead run concurrently or out of order, because the code moved on before the asynchronous operation completed. Code that depends on the result may then execute too early, using stale or default data.
- An array of Promises, such as the result of a `map()` call, is just as easy to leave unhandled as a single Promise, and is a common way to accidentally launch several operations without ever finding out if any of them failed.

This rule applies to Promise-valued statements. Promises used where another type is expected, such as in a conditional or a spread, are covered by S6544.

What is the potential impact?

An unhandled rejection is not contained to the operation that failed. Since Node.js 15, the default `--unhandled-rejections` mode is `throw`, which raises the rejection as an uncaught exception and terminates the process unless an `unhandledRejection` hook is registered. A single unobserved background task can therefore take down a server that was otherwise healthy.

■ *How can I fix it?*

Make sure every Promise-returning statement is handled in one of the following ways:

- `await` it.
- `return` it, so that the caller becomes responsible for handling it.
- Attach a rejection handler with `.catch()`, or pass a rejection handler as the second argument to `.then()`.
- If you are dealing with several Promises at once, combine them with `Promise.all()`, `Promise.allSettled()`, `Promise.race()`, or `Promise.any()` before handling the result, instead of leaving an array of Promises unhandled.

If a Promise's outcome genuinely does not matter and you want to make that explicit, use the `void` operator on the expression. This documents that the Promise is intentionally left unobserved. Note that this does not make rejections disappear: it only silences this rule, so it should be reserved for Promises that you know cannot practically reject, or that are otherwise covered by a global unhandled-rejection handler.

Noncompliant code example

■ Noncompliant Code Example:

```
1 ■ function sendMetrics(payload) { <<<
2 ■   fetch('/metrics', { method: 'POST', body: payload }); // Noncompliant: nothing handles a failed request
3 ■ }
```

✓ Compliant Solution:

```
1 ■ async function sendMetrics(payload) { <<<
2 ■   await fetch('/metrics', { method: 'POST', body: payload });
3 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs().then(logs =&gt; storeLogs(logs)); // Noncompliant: no rejection handler
3 ■ }
```

✓ Compliant Solution:

```
1 ■ function logAndForget() { <<<
2 ■   fetchLogs()
3 ■     .then(logs =&gt; storeLogs(logs))
4 ■     .catch(error =&gt; reportError(error));
5 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function notifyAll(subscribers) { <<<
```

```
2 ■   subscribers.map(subscriber =&gt; subscriber.notify()); // Noncompliant: an array of unhandled Promises
3 ■ }
```

✓ Compliant Solution:

```
1 ■ async function notifyAll(subscribers) { <<<
2 ■   await Promise.all(subscribers.map(subscriber =&gt; subscriber.notify()));
3 ■ }
```

■ Noncompliant Code Example:

```
1 ■ function onClick() { <<<
2 ■   trackEvent('button_clicked'); // Noncompliant: the Promise is left unhandled and is not marked as intentionally ignore ■
  ■ d
3 ■ }
```

✓ Compliant Solution:

```
1 ■ function onClick() { <<<
2 ■   // The analytics call is not expected to fail in a way that matters here.
3 ■   void trackEvent('button_clicked');
4 ■ }
```